PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today…

- HW2 Review
- Exercises on Loops
  - Fibonacci/Prime Number Generator
- Exercise on Random Numbers
  - Random Walk Simulator
- HW3 Hints

HW2 Review

HW2 Review

- Exercise 1: Consider three `if` clauses (you don't need `else`. why?)

```
// Exercise 1: Leap year
int year = 1582; // initialization (number doesn't matter here, but 1582 makes sense)
cout << "Enter year strictly after 1581 AD: ";
cin >> year;
cout << "This is : ";
bool leap_yr = false;
if(year % 4 == 0) {
        leap_yr = true;
}
if (year % 100 == 0) {
        leap_yr = false;
}
if (year % 400 == 0) {
        leap_yr = true;
}
if (leap_yr) {
        cout << "A leap year\n";
}
else {
        cout << "Not a leap year\n";
}
```

UCLA

HW2 Hints

- Exercise 2 (Solution 1: Strategy B from the slides in Week 4)

```
// Exercise 2: Num2Rome
int n = 1;
cout << "Enter integer
   from 1 to 3,999: ";
cin >> n;
cout << "Roman numeral: ";
while (n >= 1000) {
     cout << "M";
     n -= 1000;
}
```

```
// now n<1000
if (n >= 900) {
     cout << "CM";
     n -= 900;
}
if (n >= 500) {
     cout << "D";
     n -= 500;
}
if (n >= 400) {
     cout << "CD";
     n -= 400;
}
while (n >= 100) {
     cout << "C";
     n -= 100;
}
```

```
// now n<100
if (n >= 90) {
     cout << "XC";
     n -= 90;
}
if (n >= 50) {
     cout << "L";
     n -= 50;
}
if (n >= 40) {
     cout << "XL";
     n -= 40;
}
while (n >= 10) {
     cout << "X";
     n -= 10;
}
```

```
// now n<10
if (n >= 9) {
     cout << "IX";
     n -= 9;
}
if (n >= 5) {
     cout << "V";
     n -= 5;
}
if (n >= 4) {
     cout << "IV";
     n -= 4;
}
while (n >= 1) {
     cout << "I";
     n -= 1;
}
cout << endl;
```

Simple idea, but quite long

UCLA

HW2 Hints

- Exercise 2 (Solution 2: Strategy A from the slides in Week 4)

```
string romans = "IVXLCDM", output = "";
int place = 0;
while (n > 0) {
    if (n % 10 > 0){
        if (n % 10 == 9) { // "IX", "XC", or "CM" (02, 24, 46)
            output = romans[place*2] + (romans[place*2 + 2] + output);
            n -= 9;
        }
        else if (n % 10 == 4) { // "IV", "XL", "CD" (01, 23, 45)
            output = romans[place * 2] + (romans[place * 2 + 1] + output);
            n -= 4;
        }
        while (n % 5 > 0) { // #: 1,2,3, 6,7,8 (append I, X, C or M)
            output = romans[place * 2] + output;
            --n;
        }
        if (n % 10 == 5) { // V, L, or D
            output = romans[place * 2 + 1] + output;
            n -= 5;
        }
    }
    n /= 10;
    ++place;
}
cout << output << endl;
```

More Challenging Loop Exercises

Fibonacci/Prime Number Generator

Why are we doing this?

- There was a feedback from one of you:

> Topics you want to see in Discussion 1A *
>
> Be as specific as you can. For instance, Visual Studio tips and shortcuts, More examples on C++ arrays, HW 2 hints on some specific functionality, etc.
>
> Questions similar to those in homework and exams. p.s. I am confused about those challenging questions and I wonder how helpful they are for later programming

- Thanks for the feedback! That's a great question: how helpful are they?
- Those "challenging exercises" are in fact from HW/Quiz/Exam problems in the previous PIC10A courses
  - As you can see in the acknowledgement in the lecture slides, some instructors share their slides and HW/Exam problems

UCLA

Fibonacci/Prime Number Generator

- 1. Fibonacci Number Generator
  - The user inputs a positive integer (**n**)
  - The program outputs the **n**-th Fibonacci number F_n
  - Fibonacci numbers satisfy: $F_{n+2} = F_{n+1} + F_n$
    (c.f. rabbit population puzzle posed by Fibonacci)

- 2. Prime Number Generator
  - The user inputs a positive integer (**n**)
  - The program outputs the **n**-th prime number P_n
  - First few prime numbers are: 2, 3, 5, 7, 11, 13, …

- Input and Output format:

```
Please enter a positive integer.
5
The 5th Fibonacci number is 5.
The 5th prime number is 11.
```

UCLA

Hints

- Fibonacci numbers are relatively simple
  - Just use a for loop
  - Can be done using 3 variables

- Prime numbers is more difficult
  - Break the problem into smaller pieces
  - First create a variable `int cnt` that counts the number of primes so far
  - You can then use a while loop

```
while(cnt < n) { // we want the n-th prime…
      // increment the current number
      // check if the current number is prime
      // if so, increment cnt
      // if not, don't increment cnt
```

UCLA

Hints

- Primality test (smaller piece of the entire problem)
- An integer is not prime if it has a divisor greater than 1
- So, consider using a loop to check if it has a divisor
- *Mathematical fact: you only need to check if it has a divisor $\leq \sqrt{N}$ → how to implement this w/o using `std::sqrt()`?
- You can set a flag `bool isPrime` to store the primality test result

```
for(div = 2; div*div <= num; ++div ) {
      // check if div divides num
      // if so, set isPrime false and exit from this (inner) loop
      // if not, do nothing
```

Q: what should be the *default value* for `isPrime`?

UCLA

Exercise on Random Numbers

Random Walk Simulator

Summary for Random Numbers in C++

- `#include <cstdlib>` for `rand()` and `srand()`

- `#include <ctime>` for `time()`

- Use `time(nullptr)` to generate a seed:
  - `srand(time(nullptr));`
  - And then use `rand()`

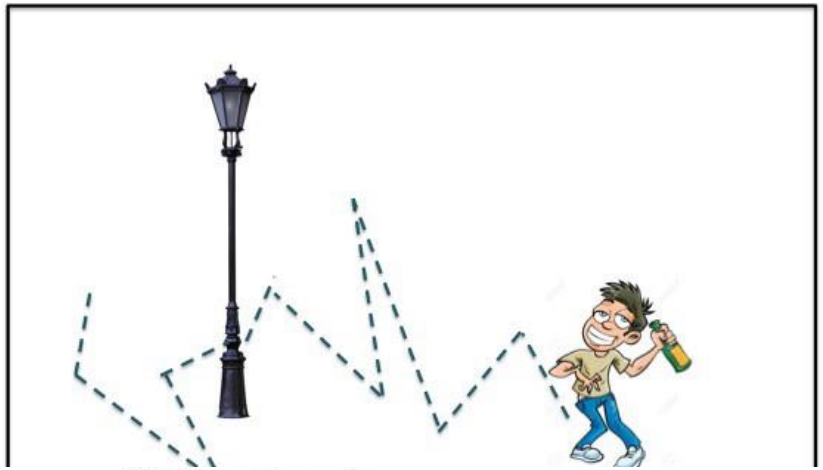
- Intervals:
  - `r_double = (rand()*1./RAND_MAX)*(B-A) + A;`
  - `r_int = A + rand()%(B-A+1);`

UCLA

Exercise – Random Walk Simulator

(will be revisited on Thursday)

- Suppose someone walks in a constant speed, but chooses the direction randomly in each step
- The path (to be more precise, this *stochastic process*) is called a random walk

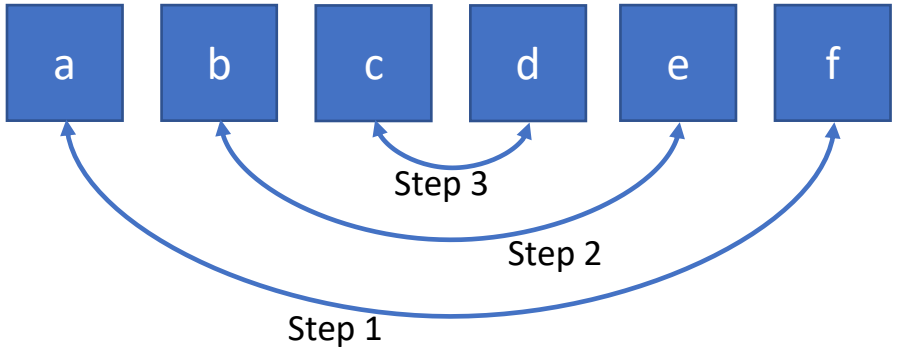


- We will simulate a 1-D random walk:
  - An object can only move forward or backward at each step
  - When the object hits the wall (upper/lower bounds), it stops walking
  - The time that it hits the wall is called the "stopping time"
  - Important in mathematical finance

HW3 Hints

HW3 Hints

- Exercise 1:
  - Consider “swapping” two characters in the string



Original string:	abcdef
after Step 1:	fbcdea
after Step 2:	fecdba
after Step 3:	fedcba

Then stop!

UCLA

HW3 Hints

- Exercise 2: Finding the 2nd smallest number in the given stream
  - Recall Grade Calculator ver.2
  - How did we find the smallest number there?
  - It should be very similar
    - Consider *two* variables instead of *one* (because we need the second smallest, and to do this you *also* need to keep track of the smallest)
    - And, add some more if's for them

  - Don't forget the same numbers don't count:
    - If the input were: "1 2 1 8 8 9 Q", the smallest and the second smallest are:
    - 1 → 1, (undecided)
    - 2 → 1, 2
    - 1 → 1, 2 (without a care, it may become "1, 1")
    - 8 → 1, 2
    - 9 → 1, 2
    - Q → quit, and output 2

Note: if the user inputs only *one* number, you should set
the second smallest = that number
(See the last test case in the instruction)

UCLA

Your Feedback is welcome

- Don’t hesitate to give a feedback on the discussion
- You can use a link in BruinLearn (Google Form)

Click this link

At this point, you may want to request
“more hw hints”,
“more challenging examples/exercises”,
“more details on the classroom examples”, etc.
Please let me know what you want!